

[Pro](#) [Teams](#) [Pricing](#) [Documentation](#)**npm**[Sign Up](#)[Sign In](#)[Search](#)

highcharts-react-official TS

3.2.1 • [Public](#) • Published 2 months ago[Readme](#)[Code](#) Beta[0 Dependencies](#)[266 Dependents](#)[21 Versions](#)

Highcharts React

Official minimal **Highcharts** wrapper for React.

Table of Contents

1. **Getting started**
 1. **General prerequisites**
 2. **Installing**
 3. **Using**
 1. **Basic usage example**
 2. **Highcharts with NextJS**
 3. **Highcharts with TypeScript**
 4. **Optimal way to update**

2. **Options details**
3. **Example with custom chart component**
4. **Get repository**
5. **Examples**
6. **Tests**
7. **Changelog**
8. **FAQ**
 1. **Where to look for help?**
 2. **Why highcharts-react-official and not highcharts-react is used?**
 3. **How to get a chart instance?**
 4. **How to add a module?**
 5. **How to add React component to a chart's element?**
 6. **Why Highcharts mutates my data?**

Getting Started

General prerequisites

Make sure you have **node**, **NPM** and **React** up to date. Tested and required versions:

- `node` 8.11.3+
- `npm` 6.4.1+ or similar package manager

This wrapper also requires `highcharts` and `react` packages with the following versions installed in your project:

For version **2.x.x** :

- `react` 16.4+
- `highcharts` 5.0.0+

For version **3.x.x** :

- `react` 16.8+
- `highcharts` 6.0.0+

Installing

Get the package from NPM in your React app:

```
npm install highcharts-react-official
```

If Highcharts is not already installed, get the package with Highcharts:

```
npm install highcharts highcharts-react-official
```

Using

Basic usage example

Import into your React project and render a chart:

```
import React from 'react'
import { render } from 'react-dom'
import Highcharts from 'highcharts'
import HighchartsReact from 'highcharts-react-official'

const options = {
  title: {
    text: 'My chart'
  },
  series: [{
    data: [1, 2, 3]
  }]
}

const App = () => <div>
  <HighchartsReact
    highcharts={Highcharts}
    options={options}
  />
</div>
```

```
render(<App />, document.getElementById('root'))
```

Highcharts with TypeScript

Live example: <https://stackblitz.com/edit/react-starter-typescript-cfcznt>

```
import React, { useRef } from 'react';
import * as Highcharts from 'highcharts';
import HighchartsReact from 'highcharts-react-official';

// The wrapper exports only a default component that at the same time
// namespace for the related Props interface (HighchartsReact.Props) and
// RefObject interface (HighchartsReact.RefObject). All other interfaces
// like Options come from the Highcharts module itself.

const options: Highcharts.Options = {
  title: {
    text: 'My chart'
  },
  series: [{
    type: 'line',
    data: [1, 2, 3]
  }]
};

const App = (props: HighchartsReact.Props) => {
  const chartComponentRef = useRef<HighchartsReact.RefObject>(null);

  return (
    <HighchartsReact
      highcharts={Highcharts}
      options={options}
      ref={chartComponentRef}
      {...props}
    />
  );
};
```

```
    />
  );
};
// Render your App component into the #root element of the document.
ReactDOM.render(<App />, document.getElementById('root'));
```

Since version 3.2.1 it is also possible to import types for `props` and `ref` independently:

```
import HighchartsReact, { HighchartsReactRefObject, HighchartsReactProc
```

Highcharts with NextJS

Next.js executes code twice - on server-side and then client-side. First run is done in an environment that lacks `window` and causes Highcharts to be loaded, but not initialized. Easy fix is to place all modules inits in a `if` checking if Highcharts is an object or a function. It should be an object for modules initialization to work without any errors, so code like below is an easy fix:

```
import React from 'react'
import Highcharts from 'highcharts'
import HighchartsExporting from 'highcharts/modules/exporting'
import HighchartsReact from 'highcharts-react-official'

if (typeof Highcharts === 'object') {
  HighchartsExporting(Highcharts)
}

...
```

This is a known issue with NextJS and is covered here:

<https://github.com/vercel/next.js/issues/5354>

Optimal way to update

A good practice is to keep all chart options in the state. When `setState` is called, the options are overwritten and only the new ones are passed to the `chart.update` method.

Live example: <https://stackblitz.com/edit/react-hketvd?file=index.js>

Optimal way to update with React Hooks: <https://stackblitz.com/edit/react-nwseym?file=index.js>

```
import React, { Component } from 'react';
import { render } from 'react-dom';
import HighchartsReact from 'highcharts-react-official';
import Highcharts from 'highcharts';

class LineChart extends Component {
  constructor(props) {
    super(props);

    this.state = {
      // To avoid unnecessary update keep all options in the state.
      chartOptions: {
        xAxis: {
          categories: ['A', 'B', 'C'],
        },
        series: [
          { data: [1, 2, 3] }
        ],
        plotOptions: {
          series: {
            point: {
              events: {
                mouseOver: this.setHoverData.bind(this)
              }
            }
          }
        }
      }
    }
  }
}
```

```
    }  
  },  
  hoverData: null  
};  
}  
  
setHoverData = (e) => {  
  // The chart is not updated because `chartOptions` has not changed  
  this.setState({ hoverData: e.target.category })  
}  
  
updateSeries = () => {  
  // The chart is updated only with new options.  
  this.setState({  
    chartOptions: {  
      series: [  
        { data: [Math.random() * 5, 2, 1]}  
      ]  
    }  
  });  
}  
  
render() {  
  const { chartOptions, hoverData } = this.state;  
  
  return (  
    <div>  
      <HighchartsReact  
        highcharts={Highcharts}  
        options={chartOptions}  
      />  
      <h3>Hovering over {hoverData}</h3>  
      <button onClick={this.updateSeries.bind(this)}>Update Series</button>  
    </div>
```

```
    )
  }
}

render(<LineChart />, document.getElementById('root'));
```

Options details

Available options with example values:

```
<HighchartsReact
  options = { this.state.chartOptions }
  highcharts = { Highcharts }
  constructorType = { 'mapChart' }
  allowChartUpdate = { true }
  immutable = { false }
  updateArgs = { [true, true, true] }
  containerProps = {{ className: 'chartContainer' }}
  callback = { this.chartCallback }
/>
```

Parameter	Type	Required	Defaults	Description
options	Object	yes	-	Highcharts chart configuration object. Please refer to the Highcharts documentation .
highcharts	Object	yes	-	Used to pass the Highcharts instance after module is initialized. If not passed, the component will throw an error.

Parameter	Type	Required	Defaults	Description
				the Highcharts framework window.
<code>constructorType</code>	String	no	<code>'chart'</code>	String for constructor method . Official constructors: - <code>'chart'</code> for charts - <code>'stockChart'</code> for Highstock charts - <code>'mapChart'</code> for Highmaps charts - <code>'ganttChart'</code> for Gantt charts
<code>allowChartUpdate</code>	Boolean	no	<code>true</code>	This wrapper uses <code>chart.update</code> method to apply options to the chart, changing the parameters of the component. This allows to turn off updating.
<code>immutable</code>	Boolean	no	<code>false</code>	Reinitialises the chart on prop update (as <code>chart.update</code> is useful in some cases, but slower than a regular update).
<code>updateArgs</code>	Array	no	<code>[true, true, true]</code>	Array of update function optional arguments. Parameters:

Parameter	Type	Required	Defaults	Descript
				should be define same order like i Highcharts funct [redraw, on animation] .I more specific de the parameters.
containerProps	Object	no	-	The props object the chart contain React.createElement method. Useful f styles or class.
callback	Function	no	-	A callback functi created chart. Fir argument for the will hold the crea chart . Default the function poin chart . This op optional.

Example with custom chart component

Create custom component `./components/MyStockChart.jsx` :

```
import React from 'react'
import Highcharts from 'highcharts/highstock'
import HighchartsReact from 'highcharts-react-official'

const options = {
  title: {
```

```
    text: 'My stock chart'
  },
  series: [{
    data: [1, 2, 3]
  }]
}

const MyStockChart = () => <HighchartsReact
  highcharts={Highcharts}
  constructorType={'stockChart'}
  options={options}
/>

export default MyStockChart
```

Render your custom chart component like below:

```
import React from 'react'
import { render } from 'react-dom'
import MyStockChart from './components/MyStockChart.jsx'

const App = () => <div>
  <MyStockChart />
</div>

render(<App />, document.getElementById('root'))
```

Get repository

Clone github repository and install dependencies:

```
git clone https://github.com/highcharts/highcharts-react
cd highcharts-react
```

```
npm install
```

Examples and tests require Highcharts library, don't forget to:

```
npm install highcharts
```

Examples

There are several interesting examples in the demo folder that use all available constructors and several modules.

Bundle these with:

```
npm run build-demo
```

Demo is located under demo/index.html

Live example: <https://stackblitz.com/edit/react-4ded5d?file=index.js>

Tests

This wrapper contains tests for: testing environment, chart rendering and passing down container props. To run tests, type:

```
npm run test
```

Changelog

The changelog is available [here](#).

FAQ

Where to look for help?

Technical support will help you with Highcharts and with the wrapper.

If you have a bug to report or an enhancement suggestion please submit **Issues** in this repository.

Why highcharts-react-official and not highcharts-react is used?

The NPM package is registered as `highcharts-react-official` because `highcharts-react` was already taken.

How to get a chart instance?

For class components and version prior to 3.0.0 use `React.createRef` :

```
constructor(props) {  
  super(props)  
  this.chartRef = React.createRef();  
}  
  
render() {  
  return (  
    <HighchartsReact  
      highcharts={ Highcharts }  
      options={ options }  
      ref={ this.chartRef }  
    />  
  );  
}
```

For functional components and version 3.0.0 and later use `useRef` hook:

```
const chartComponent = useRef(null);  
const [options] = useState({...});  
  
useEffect(() => {  
  const chart = chartComponent.current.chart;  
  ...  
}, []);
```

```
return <HighchartsReact ref={chartComponent} highcharts={Highcharts}
```

Alternatively store a chart reference in a callback function:

```
afterChartCreated = (chart) => {  
  // Highcharts creates a separate chart instance during export  
  if (!chart.options.chart.forExport) {  
    this.internalChart = chart;  
  }  
}  
  
componentDidMount() {  
  // example of use  
  this.internalChart.addSeries({ data: [1, 2, 3] })  
}  
  
render() {  
  return (  
    <div>  
      <h2>Highcharts</h2>  
      <HighchartsReact  
        highcharts={ Highcharts }  
        options={ options }  
        callback={ this.afterChartCreated }  
      />  
    </div>  
  );  
}
```

How to add a module?

To add a module, import and initialize it:

```
import Highcharts from 'highcharts'
import highchartsGantt from "highcharts/modules/gantt";
import HighchartsReact from 'highcharts-react-official'

// init the module
highchartsGantt(Highcharts);
```

alternative with `require`:

```
import Highcharts from 'highcharts'
import HighchartsReact from 'highcharts-react-official'

require("highcharts/modules/variwide")(Highcharts);
```

How to add React component to a chart's element?

By using **Portals** it is possible to add a component to every HTML chart element.

Live example: <https://codesandbox.io/s/1o5y7r31k3>

Why Highcharts mutates my data?

It can be confusing, since React props are read-only, but Highcharts for performance reasons mutates the original data array. This behaviour is NOT changed by our wrapper. You need to pass a copy of your data to the wrapper if you want to prevent mutations.

Issue: <https://github.com/highcharts/highcharts-react/issues/326>

More discussion here: <https://github.com/highcharts/highcharts/issues/4259>

Keywords

highcharts highstock highmaps react

Install

```
> npm i highcharts-react-official
```

Repository

 github.com/highcharts/highcharts-react

Homepage

 github.com/highcharts/highcharts-react#readme

Weekly Downloads

351,502



Version

3.2.1

License

MIT

Unpacked Size

240 kB

Total Files

38

Issues

2

Pull Requests

0

Last publish

2 months ago

Collaborators



[Try on RunKit](#)

[Report malware](#)